

AI编程实践：从Claude Code实践到团队协作的优化思考 | 得物技术

 mp.weixin.qq.com/s/GjPUDjxx2XmZX-G5qDC6eA

目录

一、开发痛点：为什么我们需要AI编程辅助？

二、Claude Code核心功能解析：从工具到方法论

1.精准对话流设计：控制AI思考的艺术

2.Plan模式：复杂任务的系统化分解

3.系统提示词：给AI立"规矩"的艺术

4.SKILL与MCP：知识沉淀与外部能力扩展

三、对话流设计方法论：让AI"懂"你的真实需求

1.现状分析：传统对话模式的局限性

2.核心问题：为什么AI总是"听不懂"？

3.解决方案：结构化对话设计方法

4.为什么这样设计：背后的认知科学原理

四、AI团队协作模式：子代理系统的实践与思考

1.团队协作的现状与挑战

2.Claude Code的子代理协作模式

3.子代理协作的价值与局限

4.子代理协作的设计思考

五、实践经验与未来展望

1.实践经验总结

2.AI编程的局限性认知

3.未来发展方向思考

六、结语：人机协作的新型开发范式

开发痛点：为什么我们需要AI编程辅助？

核心发现：AI编程工具正在重塑开发流程，但真正的价值不在于替代开发者，而在于构建人机协作的新型开发范式。Claude Code通过精准对话流设计、模块化任务分解和专业化子代理协作，在提升开发效率的同时，也面临着上下文管理、协作边界和质量控制等实际挑战。

作为一线开发者，我们每天都在与复杂的业务逻辑和不断迭代的技术栈打交道。不知道你是否也遇到过这些场景：刚理清一个复杂业务流程，被打断后又得重新梳理思路；接手一个老项目，花了半天还没搞懂其中某个模块的设计思路；或者在不同项目间切换时，总要重新适应不同的编码规范和架构风格。

日常开发的三个“拦路虎”：

- **上下文切换成本高：**需求理解→技术选型→代码实现→质量验证的切换过程中，每次都要重新构建认知框架。
- **知识传递效率低：**项目规范、架构经验分散在文档和个人经验中，新成员上手或跨模块开发时处处碰壁。
- **开发流程割裂：**需求→设计→编码→审查各环节串行传递，信息易失真且反馈滞后。

这些问题不是简单的“加人”或“加班”能解决的。我们需要的是一种新的开发范式，而Claude Code这类AI编程工具正是在这样的背景下进入了我们的视野。它的价值不在于替我们写代码，而在于成为我们的“认知放大器”和“流程协作者”。

二

Claude Code核心功能解析：从工具到方法论

Claude Code构建了一套完整的AI辅助开发方法论。接下来将结合团队实际使用经验，从功能特性、使用场景和设计初衷三个维度，详细介绍其核心功能：

精准对话流设计：控制AI思考的艺术

第一次用Claude Code时，就像面对一个热情但经验不足的实习生——如果不明确告诉他要做什么、怎么做、有什么要求，他很可能会给你一个“惊喜”。对话流设计就是解决这个问题的关键。

设计初衷：对话流设计的本质是将人类的编程思维模式转化为AI可理解的结构化交互方式，通过明确的上下文管理和约束条件设置，引导AI生成符合预期的代码结果。

核心功能

对话流设计通过三个关键机制控制AI的思考过程：

- **上下文聚焦：**要求单次对话仅处理一个功能模块，避免多任务混合导致的AI注意力分散。我们曾经试过在一个对话里同时让AI处理多个模块，结果它把两个模块的错误处理逻辑混在了一起。
- **约束明确化：**通过具体指令减少AI的自由度，比如"仅修改X包下文件"、"必须复用Y工具类"。这些约束要尽可能具体，比如不说"遵循项目规范"，而是说"使用ResultDTO作为统一返回格式，错误码规则参考ErrorCodeEnum"。
- **增量式提问：**采用"先框架后细节"的提问策略，先让AI生成接口定义和整体框架，待确认后再逐步深入实现细节。这种方式很像我们带新人时"先搭骨架再填肉"的指导方法。

使用心法

启动新功能开发时，我们会创建专用对话线程，并在初始prompt中明确四件事：

1. 当前任务的功能边界和目标（做什么，不做什么。）
2. 必须遵守的技术约束和规范（用什么技术栈，遵循什么标准。）
3. 期望的输出格式和交付物（要代码？要文档？还是两者都要？）
4. 分阶段的实现计划（先设计接口，再实现逻辑，最后写测试。）

真实踩坑经验

处理跨模块依赖时，我们发现AI很容易"忘记"之前设定的约束。后来我们总结出一个技巧：每开始一个新的实现阶段，就简要回顾一下关键约束。比如："现在我们要处理任务交接流程，请记得：1. 使用Redis分布式锁；2. 需要修改商运关系和新商成长任务；3. 异常处理要符合规范。"

Plan模式：复杂任务的系统化分解

面对"实现一个完整的拜访任务系统"这样的复杂需求，直接让AI生成代码就像让一个刚入行的开发者独立负责整个项目——结果往往是逻辑混乱、漏洞百出。Plan模式就是解决这个问题的"项目管理工具"。

现状与问题：我们早期使用Claude Code时，经常犯一个错误：把一个复杂需求一股脑丢给AI，然后期待它给出完美解决方案。结果通常是：

- 实现逻辑不完整，有些边界情况根本没考虑到。

- 模块间接口设计不一致，调用起来磕磕绊绊。

- 技术选型不合理，用了个"看起来很酷但项目中并不适用"的方案。

这其实不能怪AI，人在面对过于复杂的问题时也会手足无措。我们需要一种方法把大问题拆分成小问题，而Plan模式正是借鉴了项目管理中的WBS（工作分解结构）思想。

我们的解决方案：Plan模式使用三步法分解需求：

1. 需求分析与模块划分：把整体需求分解为独立的功能模块。

2. 技术方案设计：为每个模块确定实现思路和技术选型。

3. 任务优先级排序：根据依赖关系和重要性确定实现顺序。

实际操作示例：拜访任务需求

拜访任务是一个简单的企业内部任务需求，在实现"拜访任务需求"时，我们通过Plan模式生成的任务清单是这样的：

---拜访任务线上化 - 模块清单M1: 任务创建模块- 功能: 创建拜访任务,包含基本信息、拜访对象、参与人员- 复杂度: Medium (需要处理多表关联和事务)M2: 任务审批模块- 功能: 飞书审批流程集成,支持任务提交、审批、驳回- 复杂度: High (涉及飞书审批接口集成和状态流转)M3: 日程同步模块- 功能: 将拜访任务同步到飞书日历- 复杂度: Medium (飞书API调用和异常处理)M4: 任务列表查询模块- 功能: 分页查询任务列表,支持多维度筛选- 复杂度: Medium (ES聚合查询和数据转换)M5: 任务详情查询模块- 功能: 查询任务详情,包括拜访对象、参与人员、商家信息- 复杂度: Low (简单的关联查询)M6: 任务统计模块- 功能: 统计不同类型、不同状态的任务数量- 复杂度: Low (聚合统计)M7: 拜访结果提交模块- 功能: 提交拜访记录,上传附件,记录拜访内容- 复杂度: Medium (文件上传和多表操作)M8: 任务分配模块- 功能: 批量分配任务给运营人员- 复杂度: High (涉及分配策略和定时任务)M9: 任务触达模块- 功能: 任务开始、结束、上传提醒通知- 复杂度: Medium (飞书消息卡片模板)M10: ES数据同步模块- 功能: 将任务数据同步到ES,支持全文检索- 复杂度: Medium (事件驱动和ES批量操作)M11: 任务导入模块- 功能: Excel批量导入拜访任务- 复杂度: High (Excel解析、数据校验、批量插入)M12: 任务状态变更模块- 功能: 定时更新任务状态(未开始→进行中→已结束)- 复杂度: Low (定时任务和批量更新)---第二步:技术方案设计| 模块 | 数据存储 | 查询方案 | 外部集成 | |-----|

MySQL (事务) | - | - || M1-任务创建 |
审批记录表 | - | 飞书审批API || M2-任务审批 | MySQL + 审批记录表 | -
| - | 飞书日历API || M3-日程同步 | -
查询 | - || M4-任务列表 | ES (主) + MySQL (辅) | ES聚合
| 商家信息API || M5-任务详情 | MySQL | 联表查询
| M6-任务统计 | ES | ES聚合统计 | -
|| M7-结果提交 | MySQL + OSS | - | OSS文件上传 || M8-任务分配 | MySQL | 分配算法 | -
| - | 飞书消息卡片 || M9-任务触达 | -
| Spring Event | - || M10-ES同步 | ES
| Excel 解析 | - || M11-任务导入 | MySQL
| - || M12-状态变更 | MySQL | JOB定时任务

---第三步:任务优先级排序 核心流程(必须先实现)1. M1-任务创建 (基础功能)2. M5-任务详情 (数据展示)3. M4-任务列表 (核心查询)4. M7-结果提交 (闭环流程)P1 审批与通知(核心功能增强)5. M2-任务审批 (依赖M1)6. M3-日程同步 (依赖M2审批通过)7. M9-任务触达 (依赖M1、M7)P2 运营功能(效率提升)8. M8-任务分配 (依赖M1)9. M11-任务导入 (依赖M1)10. M6-任务统计 (依赖M4)P3 技术优化(性能优化)11. M10-ES同步 (依赖M1、M4)12. M12-状态变更 (独立模块)---实现建议阶段一:基础能力- 先实现M1、M5、M4,打通任务创建→查询→详情的基本链路- 使用 MySQL 实现,暂不接入 ES阶段二:审批流程- 实现 M2、M3,接入飞书审批和日历- 完成任务的完整生命周期管理阶段三:结果闭环- 实现 M7,完成拜访结果提交- 实现 M9,添加任务触达通知阶段四:运营优化- 实现 M8、M11、M6- 提升运营效率阶段五:性能优化(按需)- 实现 M10,将查询迁移到 ES- 实现 M12,优化任务状态管理

这种方式不仅让AI能够有序工作，也让我们对整个开发进度有了更清晰的把控。

系统提示词：给AI立"规矩"的艺术

如果把Claude Code比作一个新加入团队的开发人员，系统提示词（CLAUDE.md）就相当于给他的"入职手册"，告诉他团队的编码规范、工作流程和注意事项。

新手常犯的错误：把系统提示词写成"百科全书"，恨不得把所有项目知识都塞进去。结果AI要么忽略大部分内容，要么在生成代码时顾此失彼。我们早期的系统提示词长达5000字，包含了从架构设计到代码规范的所有内容，效果反而不好。

实践心得：有效的系统提示词应该像"护栏"而非"详尽手册"。我们发现，针对AI常见错误模式设计的针对性提示，远比全面但泛泛的规范更有效。现在我们的系统提示词控制在200字以内，只包含最关键的约束和指引。

系统提示词模板

经过多次迭代，我们总结出包含三个关键模块的系统提示词结构：

模块	内容示例	目的
角色定义	"你是商家域订单系统的开发专家，采用增量迭代的开发模式，负责逐个模块的开发-审查-验证闭环管理。"	明确AI在当前任务中的身份定位
能力边界	"你可自主生成单元测试和基础CRUD代码，但涉及数据库表结构变更、跨服务接口设计必须获得人工确认。"	界定AI的自主决定权范围
输出规范	"代码必须包含完整注释（类注释、方法注释、关键逻辑注释），复杂业务逻辑需要添加流程图说明。"	标准化AI的输出格式

使用技巧

分享几个在实践中总结的系统提示词编写技巧：

- **避免信息过载**：不要试图包含所有知识，而是指引AI在需要时查询特定文档。例如："遇到分布式事务问题时，请参考/doc/分布式事务最佳实践.md文档中的TCC模式实施方案"。
- **提供正向引导**：不仅说"不要做什么"，更要明确"应该怎么做"。例如，不说"不要使用过时的API"，而说"请使用OrderServiceV2替代OrderServiceV1"。
- **动态调整策略**：我们每两周会回顾一次系统提示词的有效性，根据AI最近常犯的错误补充新的约束。比如发现AI经常忘记处理空指针，就新增一条："所有方法入参必须进行非空校验，使用ValidateUtil.isEmpty()方法，异常时抛出IllegalArgumentException"。

SKILL与MCP：知识沉淀与外部能力扩展

在团队协作中，我们经常说"不要重复造轮子"。同样，在使用Claude Code时，我们也需要一种机制来沉淀和复用那些有效的Prompt和解决方案——这就是SKILL和MCP机制的价值所在。

SKILL机制：把好经验变成"可复用组件"

SKILL本质上是将单次生效的Prompt指令沉淀为可反复调用的标准化复用资产。举个例子，我们团队处理"ES数据查询"逻辑时，总结出了一个内部版本的SDK。我们把这个SDK的调用方式封装成一个SKILL，以后遇到类似场景，只需调用这个SKILL，AI就能按照我们团队的最佳实践来实现。

MCP协议：让AI能"调用"外部工具

MCP（模型上下文协议）解决了AI与外部工具、数据源的连接问题。通过MCP，AI不再局限于静态知识，而是能够动态访问实时数据。我们集成了飞书MCP服务器，让AI能够直接操作飞书平台，如自动生成技术方案文档、读取PRD需求、同步数据到多维表格等。

最适合封装为SKILL的场景

- 1.复杂工具使用指南：如"ElasticSearch接入"、"Redis缓存更新策略"等需要特定知识的场景。
- 2.常见错误处理模板：如"分布式锁冲突处理"、"数据库乐观锁重试机制"等反复出现的问题解决方案。

MCP协议的典型应用场景

场景1: 自动生成技术方案文档

AI分析需求后，通过飞书MCP调用feishu_create_doc；

直接在指定的知识库目录创建格式化的技术方案文档；

省去手动复制粘贴的繁琐步骤。

场景2: 读取PRD需求

用户提供飞书文档链接；

AI通过feishu_get_doc_content获取文档内容；

基于完整需求信息生成技术方案和实现计划。

场景3: 数据同步到多维表格

代码生成后的统计数据(如代码行数、涉及文件等)；

通过feishu_append_bitable_data自动追加到飞书多维表格；

便于团队追踪AI编程效率指标。

三

对话流设计方法论：让AI"懂"你的真实需求

刚接触Claude Code时，我们采用的是简单直接的"需求-响应"模式：开发者描述需求，AI生成代码，开发者修改调整。这种模式在处理简单功能时还行，但遇到复杂场景就会出问题。

现状分析：传统对话模式的局限性

我们早期在项目中踩过的三个坑：

三大典型问题：

需求表达不完整：

开发者说"实现一个商家信息查询接口", AI生成了基础的CRUD代码, 但没有考虑商家数据权限、数据脱敏、缓存策略等实际业务需求 ;

实现任务时, 只描述了"需要任务分配功能", 结果AI生成的代码没有处理任务池、任务优先级、分配策略等核心逻辑。

上下文管理混乱 :

一个对话持续了十几轮后, AI开始忘记我们前面确定的"使用MyBatis-Plus + BaseMapper"的设计决策, 擅自改成了JPA Repository模式 ;

在实现相关功能时, 早期确定的DTO转换规范在后续模块中被遗忘, 导致代码风格不一致。

迭代反馈滞后 :

等AI生成完整的Service + Controller + Repository代码后才发现方向不对, 比如数据库表设计与现有架构冲突, 不得不从头再来, 浪费了大量时间 ;

实现触达功能时, 生成的飞书消息发送代码没有考虑现有的FeishuClient封装, 重复造了轮子。

核心问题 : 为什么AI总是"听不懂" ?

深入分析后, 我们发现传统对话模式失败的根源在于三个核心矛盾 :

语义鸿沟

自然语言描述的模糊性与代码逻辑的精确性之间的差距。我们说"这个接口要安全", AI可能理解为"需要登录校验", 而我们实际想要的是 :

- 使用项目中的@Permission注解进行权限校验。
- 参数需要使用ValidatorUtil进行校验。
- 敏感操作需要记录操作日志。

约束衰减

随着对话推进, 早期设定的技术约束在AI理解中的权重逐渐降低。就像我们记笔记时, 重要的事情要反复强调。比如 :

- 第1轮对话强调"必须继承BaseServiceImpl"。
- 第5轮对话AI可能忘记这个约束, 直接实现了一个独立的Service类。
- 第10轮对话可能连项目的分层架构都混淆了。

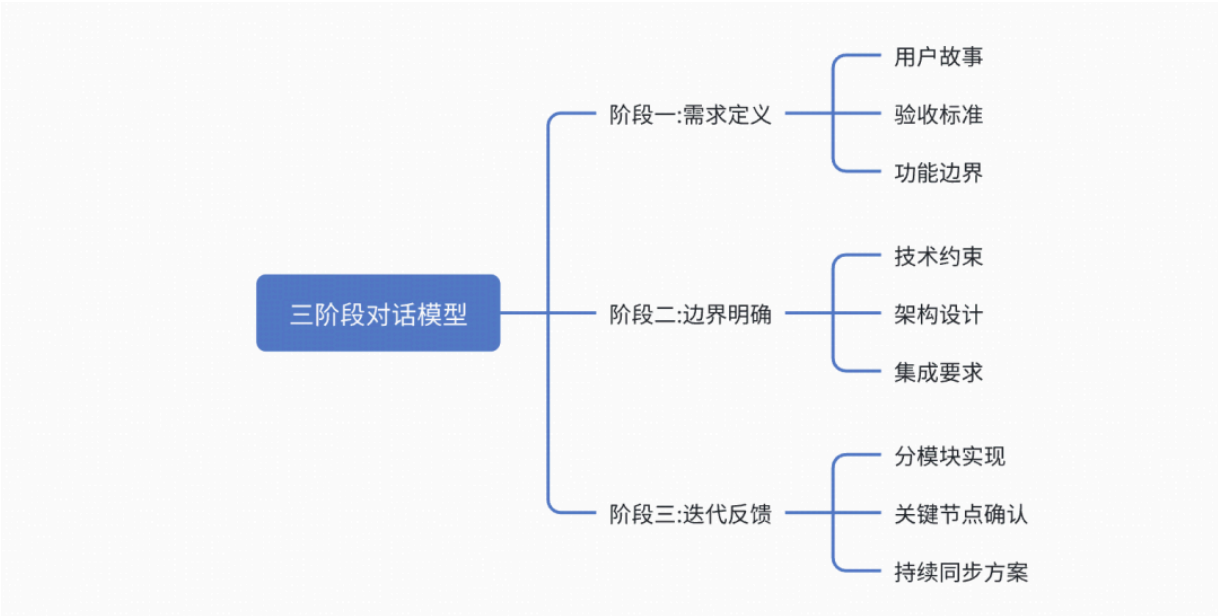
目标偏移

在 多轮对话中，AI 容易过度关注当前细节而忽视整体目标。比如讨论某个接口的参数设计时：

- AI 可能会纠结于参数名称是否优雅。
- 而忽略了这个接口的核心业务价值是"快速检索符合条件的商家"。
- 结果生成的代码参数命名很完美，但缺少了分页、排序等实际必需的功能。

解决方案：结构化对话设计方法

针对这些问题，我们团队总结出一套"三阶段对话模型"，现在已经成为我们使用Claude Code的标准流程：



阶段一：需求定义——把"要做什么"说清楚

这个阶段的目标是确保我们和AI对需求达成共识。我们会用"用户故事+验收标准"的格式来描述需求：

示例1：新商户成长任务分配

【用户故事】作为新商户运营，我需要一个任务分配功能，以便将成长任务高效分配给运营人员【验收标准】 - 支持从任务池中按优先级(P0/P1/P2)筛选待分配任务 - 支持指定运营人员进行任务分配，需校验运营人员是否有权限 - 分配时需检查运营人员当前任务负载，超过上限时提示"当前任务数已达上限" - 分配成功后需发送飞书消息通知运营人员，消息内容包含任务详情和截止时间 - 操作需记录到表，包含操作人、操作时间、任务ID、分配对象

示例2：商家数据权限查询

【用户故事】作为商家运营，我需要一个商家信息查询接口，查询结果需要根据我的数据权限进行过滤【验收标准】 - 支持按商家ID、商家名称、商家状态进行查询 - 支持分页查询，默认每页20条，最大100条 - 查询结果需要根据当前用户的数据范围进行过滤 - 商家敏感信息(手机号、身份证号)需脱敏处理 - 接口需要权限校验，至少具有"商家查看"权限 - 查询条件需记录到操作日志，便于审计

阶段二：边界明确——确定"怎么做"的约束条件

在这个阶段，我们会明确技术栈选择、架构设计和各种约束条件。关键是要区分"必须遵守"和"建议参考"的约束：

示例1：新商户成长任务模块

【技术约束】必须遵守： - 使用SpringBoot标准分层架构,所有Service继承OcsBaseServiceImpl - 数据库操作使用MyBatis-Plus,实体类继承BaseEntity,Mapper继承BaseMapper - 接口返回统一使用Result<T>格式,错误码使用ErrorCode - 权限校验使用@Permission注解,参数校验使用@Valid + ValidatorUtil - 飞书消息发送必须使用FeishuClient,不要重复实现
建议参考： - 任务状态流转参考TaskServiceImpl中的状态机模式 - 批量分配操作参考AssignImportHandler中的异步处理方式 - 运营人员权限校验参考OperatorRelationServiceImpl - 数据权限过滤参考ScopeServiceImpl中的范围查询逻辑
【数据库约束】 - 新增表必须包含created_at, updated_at, is_deleted字段 - 表名使用ocs_前缀,字段名使用蛇形命名法 - 索引设计需考虑查询场景,高频查询字段必须建立索引 - 外键约束通过代码层面维护,不在数据库层面创建

示例2：机器人问答功能

【技术约束】必须遵守： - Controller层使用@RestController + @RequestMapping,路径遵循/api/v1/{module}/{action}格式 - Service层业务逻辑必须有事务控制,使用@Transactional(rollbackFor = Exception.class) - DTO转换使用项目中的ConvertUtil,不要手动赋值 - 第三方API调用(如Dify)必须有重试机制和降级策略 - 敏感配置(API Key)必须从配置中心读取,不要硬编码
建议参考： - 对话上下文管理参考RobotServiceImpl中的会话ID生成逻辑 - 消息发送失败重试参考FeishuMessageHandler中的重试策略 - 错误处理参考GlobalExceptionHandler中的统一异常处理
【集成约束】 - Dify API调用统一通过DifyClient,响应格式需转换为项目标准格式 - 飞书机器人回调处理需实现签名校验,参考FeishuEventHandler - 所有外部API调用需记录请求日志到integration_log表

阶段三：迭代反馈——在"做的过程"中持续对齐

这个阶段的核心是增量验证，避免一次性生成大量代码后才发现方向错误。

实践要点：

分模块实现，逐个验证

- 先实现核心接口(如任务分配)，验证通过后再实现辅助功能(如任务查询)。
- 先实现Service层核心逻辑，验证业务流程正确后再补充Controller和参数校验。

关键节点主动暂停

- 数据库表设计完成后，先让开发者确认表结构再继续。
- 核心业务逻辑实现完成后，先通过单元测试验证再生成Controller。
- 集成第三方API前，先确认调用方式和参数映射。

持续同步技术方案

- 发现架构设计与现有代码不一致时，立即更新tech-solution.json和tech-solution.md。
- 新增数据表、修改核心逻辑时，同步更新技术方案文档。
- 每个模块完成后，更新模块状态为implemented → reviewed → verified。

对话示例：

开发者："实现新商户成长任务分配功能"
AI："我将分三步实现：1. 先设计数据库表结构(ocs_task_assignment表)2. 实现核心分配逻辑(NewMerchantTaskAssignService)3. 实现Controller层接口
现在开始第一步,设计的表结构如下:[展示表结构]请确认表结构是否符合需求?"
开发者："字段没问题,但索引需要加上(operator_id,status)"
AI："已更新索引设计,现在开始实现Service层核心逻辑...[生成Service代码]
核心逻辑已实现,包含:-任务池查询(按优先级过滤)-运营人员负载检查-任务分配和状态更新-飞书消息发送
请验证业务逻辑是否正确?"
开发者："逻辑正确,继续实现Controller"
AI："开始实现Controller层...[生成Controller代码]已完成模块M1实现,更新技术方案状态为implemented"

对话设计三原则与常见问题应对策略

问题类型	表现	我们的解决方案	背后思路
需求过载	AI返回概要而非具体实现	拆分为单功能对话	控制认知负荷在合理范围内
上下文漂移	忘记早期约束	每轮对话开始简要重申关键约束	强化重要信息在对话中的权重
技术栈误解	使用错误框架API	提供1-2行示例代码	具体示例比抽象描述更有效
幻觉生成	虚构不存在的方法	提供真实类定义或接口文档	基于事实的约束比禁止性约束更有效

为什么这样设计：背后的认知科学原理

这种结构化对话设计不是凭空想出来的，而是基于我们对人类认知过程的理解：

- **工作记忆限制理论**：就像我们一次只能记住7±2个信息块一样，AI的上下文理解能力也是有限的。通过分阶段对话和单次聚焦单模块，我们控制了每次交互的认知负荷。
- **渐进式知识构建**：学习和理解是一个渐进过程，先掌握整体框架再深入细节，符合认知规律。这和我们教新人时"先讲架构图，再讲模块间交互，最后讲具体实现"的思路是一致的。

四

AI团队协作模式：子代理系统的实践与思考

随着团队使用Claude Code的深入，我们发现单个AI助手已经难以满足复杂项目的开发需求——就像一个人再厉害也干不了一个团队的活。于是，我们开始探索让多个AI"角色"协同工作的模式，这就是子代理（SubAgent）系统的由来。

团队协作的现状与挑战

在传统开发模式中，我们有需求分析师、架构师、开发工程师、测试工程师等不同角色，他们通过文档、会议和代码审查等方式协作。这种模式虽然成熟，但在快节奏的业务迭代中，我们发现了一些问题：

协作中的三大痛点：

- **信息传递损耗**：需求文档从产品经理到开发再到测试，每经过一个环节就可能产生一些理解偏差。就像玩“电话游戏”，信息传到最后可能已经面目全非。
- **责任边界模糊**：当出现问题时，有时会出现“这是架构设计问题”、“这是实现问题”、“这是测试不充分”的互相推诿。
- **反馈周期漫长**：从需求分析到代码审查，整个流程走下来往往需要几天时间，等发现问题时可能已经投入了大量开发资源。

这些问题促使我们思考：能不能在Claude Code中模拟团队协作模式，让不同的AI角色各司其职又协同工作？

Claude Code的子代理协作模式

借鉴了MetaGPT等框架的思想，我们在Claude Code中构建了由多个专业化子代理组成的AI团队协作系统。每个子代理承担特定角色，通过标准化中间产物协同工作。

核心工作机制：中间产物驱动

所有子代理通过共享“技术方案文档”进行协作，这个文档就像团队的“共享白板”，包含需求分析、模块划分、实现状态和接口设计等关键信息。每个子代理只负责修改文档中与自己角色相关的部分，确保信息一致性。

四个核心子代理角色

技术方案架构师

负责需求分析、技术方案设计和模块划分。相当于团队里的架构师，输出“技术方案文档”这个“施工蓝图”。

核心职责：

- 需求拆解与模块划分
- 技术栈选型与架构设计
- 接口定义与数据模型设计
- 模块间依赖关系梳理
- 技术方案文档编写与维护

代码审查专家

负责代码质量审查。扮演技术负责人的角色，从架构合规性、代码规范和稳定性等角度挑毛病。

核心职责：

- 检查代码是否符合架构设计
- 验证代码规范和命名约定
- 识别潜在性能问题和bug
- 评估代码可维护性和扩展性
- 提供具体修改建议

代码实现专家

专注于代码实现和单元测试编写。就像主力开发工程师，按照架构师设计的蓝图一块块地实现功能。

核心职责：

- 根据技术方案实现代码
- 编写单元测试和集成测试
- 修复代码审查中发现的问题
- 编写API文档和使用说明
- 同步更新技术方案实现状态

前端页面生成器

专门负责生成符合我们低代码平台规范的前端页面配置。这是针对我们商家域管理后台特点定制的角色。

核心职责：

- 根据接口定义生成前端页面配置
- 实现表格、表单、详情页等标准组件
- 配置页面权限和数据范围过滤
- 优化前端交互体验
- 确保符合设计规范和响应式要求

协作流程

我们采用"先整体规划，再迭代实现"的工作方式，有点像敏捷开发中的Sprint规划+Daily Scrum：

1. 整体规划阶段：

- 产品经理提供需求文档。
- 协调者调用"技术方案架构师"子代理分析需求，生成技术方案文档。
- 团队评审技术方案，提出修改意见。
- 架构师子代理根据反馈修改方案，直到团队确认。

2. 单模块迭代阶段：

- 协调者从技术方案文档中选取一个模块。
- 调用"代码实现专家"生成代码。
- 调用"代码审查专家"审查代码。
- 实现专家根据审查意见修改代码。
- 重复"实现-审查-修改"直到通过。
- 更新技术方案文档，标记该模块为"已完成"。
- 进入下一个模块。

子代理协作的价值与局限

实践中的三个显著价值

- **专业化分工提升质量**：每个子代理专注于特定领域，就像专科医院比综合医院在特定疾病上更专业一样。我们发现，专门的代码审查子代理比通用AI能发现更多潜在问题。
- **流程标准化降低风险**：通过技术方案文档和明确的角色分工，开发流程被标准化和可视化。新人加入项目时，只要看技术方案文档就能快速了解整体情况。
- **知识沉淀促进复用**：子代理的专业知识和决策逻辑被编码为可复用的配置和规则，避免了"人走经验丢"的问题。

遇到的四个实际挑战

子代理协作的挑战与应对：

- **上下文同步问题**：当技术方案文档更新时，各子代理有时不能立即同步最新信息。解决办法：每次修改文档后，明确通知相关子代理"技术方案中XX部分已更新"。

- **协作边界模糊**：在处理跨模块功能时，出现“该由哪个子代理负责”的困惑。解决办法：在技术方案文档中添加“责任人”字段，明确每个模块由哪个子代理负责。
- **灵活性与标准化的平衡**：高度标准化的流程有时会限制处理特殊情况的灵活性。解决原则：90%的常规情况严格遵循标准流程，10%的特殊情况由人工介入处理。
- **错误传递放大效应**：如果技术方案设计阶段就有问题，这个问题会在后续实现和审查阶段被放大。解决办法：加强技术方案的人工评审环节，确保“地基”打牢。

子代理协作的设计思考

在设计这套协作模式时，我们有几个关键思考：

- **为什么选择“中间产物驱动”而非“直接沟通”？**

直接让子代理之间对话可能更灵活，但会导致沟通成本指数级增加（ n 个代理就有 $n(n-1)/2$ 种沟通渠道）。通过“技术方案文档”这个单一事实来源，我们大大降低了协作复杂度，也便于追踪变更历史。

- **角色划分的依据是什么？**

我们的角色划分基于软件开发的自然阶段（设计→实现→审查）和专业领域（后端→前端），这符合软件开发生命周期的自然规律。没有盲目追求角色数量，而是根据实际需求逐步增加。

- **为什么采用“增量迭代”而非“一次性开发”？**

复杂系统的构建本质上是一个不断学习和调整的过程。增量迭代让我们能够及早发现问题并调整方向，避免在错误的道路上走得太远。这和我们常说的“小步快跑，快速迭代”理念一致。

五

实践经验与未来展望

经过几个月的Claude Code实践，从最初的“试试看”到现在成为离不开的开发工具，我们积累了一些经验，也对AI编程的未来有了更清晰的认识。

实践经验总结

人机协作的最佳平衡点：

我们发现最有效的AI编程模式是“人类主导，AI辅助”，而不是反过来。我们将工作内容分为三类：

- **AI主导**：标准化代码生成（如基础CRUD接口）、单元测试编写、API文档生成等重复性高、规则明确的任务。
- **人机协作**：技术方案设计、复杂逻辑实现、代码审查等需要结合领域知识和创造性思维的任务。
- **人类主导**：需求分析、架构设计、质量决策等高风险、高创造性的任务。

上下文管理的实用技巧

管理好对话上下文是用好Claude Code的关键，分享几个我们团队总结的技巧：

- **对话线程化**：为不同功能模块创建独立对话线程。我们曾经在一个对话里讨论三个不同模块，结果上下文混乱到不得不从头开始。
- **关键信息锚定**：重要的技术决策和约束要在对话中反复强调。就像写文章时，核心观点要多次出现。
- **文档外化**：复杂设计和决策要记录在外部文档中，而不是仅依赖对话历史。我们会在对话中引用这些文档："数据库设计详见/doc/db_design.md，特别是索引设计部分"。
- **状态可视化**：通过技术方案文档中的进度标记（如[未开始]、[设计中]、[已实现]、[已审查]），直观跟踪开发状态。

质量控制的三个关键策略

使用AI生成代码后，质量控制变得更加重要。我们的做法是：

- **多层次验证**：单元测试（AI生成）+ 集成测试（人工设计）+ 代码审查（人机结合）的三层验证体系。
- **渐进式信任**：从简单、低风险模块开始使用AI，建立信任后再逐步扩展。我们最先用AI生成内部工具，验证没问题后才用于核心业务系统。
- **错误模式学习**：记录AI常犯的错误类型，针对性优化系统提示词。我们有一个"AI错误案例库"，记录了"AI忘记处理分布式锁超时"、"日期格式转换错误"等典型问题及解决方案。

AI编程的局限性认知

在实践过程中，我们也清醒地认识到AI编程并非万能解决方案，它有几个明显的局限性：

- **创造性思维不足**：AI擅长在已有知识范围内进行组合和优化，但在需要突破性创新的场景下表现有限。比如我们尝试让AI设计一个全新的商家结算模型时，它还是会倾向于参考现有模型进行修改，难以跳出固有思维框架。

- **上下文理解深度有限**：尽管Claude Code的上下文窗口已经很大，但对于我们系统中某些“牵一发而动全身”的核心模块，AI还是难以把握其深层设计意图和与其他模块的隐性依赖。
- **质量责任边界模糊**：当AI生成的代码出现质量问题时，责任界定变得复杂。我们的解决办法是：开发者对AI生成的代码负全部责任，就像我们对自己写的代码负责一样。
- **领域知识滞后性**：AI对我们公司内部系统的最新变更反应不够及时。为此我们建立了“知识库更新机制”，每月将最新的系统变更和业务规则整理成文档，供AI参考。

未来发展方向思考

基于这些实践经验，我们对AI编程工具的未来发展有几点思考：

- **更智能的上下文管理**：未来的AI编程工具应该能自动识别相关上下文、追踪依赖关系，并在适当的时候提醒开发者潜在的上下文冲突。就像经验丰富的团队领导，能记住每个人负责的模块和项目的整体情况。
- **多模态交互模式**：除了文本对话，未来可能引入图表、流程图等多种交互方式。有时画一个简单的流程图(PlantUML)，比写几百字描述更能说明问题。
- **自适应学习机制**：AI编程工具应该能从团队的使用反馈中学习，适应特定团队的编码风格和业务领域。就像新加入团队的开发者，会逐渐适应团队的工作方式。

六

结语：人机协作的新型开发范式

回顾这几个月使用Claude Code的经历，我们最大的体会是：AI编程工具的价值不在于替代开发者，而在于构建人机协作的新型开发范式。在这种范式下，人类开发者从繁琐的重复劳动中解放出来，更专注于需求分析、架构设计和质量把控等高价值创造性工作，而AI则承担起代码实现、文档生成和基础验证等标准化工作。

Claude Code作为我们实践的核心工具，通过精准对话流设计、模块化任务分解和专业化子代理协作，展示了这种新型开发范式的潜力。但我们也认识到，成功的AI编程应用需要“工具+方法论+团队协作”三位一体的系统性变革，其中人的角色从“代码生产者”向“问题解决者”和“质量把控者”转变。

作为开发者，我们需要保持开放学习的心态，积极探索和适应这种新范式。未来已来，与其恐惧被AI替代，不如学会与AI协作，在人机协作中实现更高的个人价值和团队效能。毕竟，代码只是解决问题的手段，而非目的；AI只是增强我们能力的工具，而真正的创新和价值，始终源于人的智慧和创造力。

实践启示：在AI编程时代，最有价值的开发者不是"写代码最快的人"，而是"最会引导AI、最能把控质量、最能解决复杂问题的人"。掌握与AI协作的技巧，建立系统化的AI辅助开发流程，将成为未来开发者的核心竞争力。我们的经验表明，通过合理设计对话流程、明确分工协作和严格质量控制，AI编程工具能够显著提升团队效能，但这需要整个团队在思维方式和 workflows 上的共同转变。

往期回顾

1. [入选AAAI-PerFM | 得物社区推荐之基于大语言模型的新颖性推荐算法](#)
2. [Galaxy比数平台功能介绍及实现原理 | 得物技术](#)
3. [得物App智能巡检技术的探索与实践](#)
4. [深度实践：得物算法域全景可观测性从 0 到 1 的演进之路](#)
5. [前端平台大仓应用稳定性治理之路 | 得物技术](#)